Übungsblatt 9: Klausurtraining zu Vorlesung 1–7

Mikrocomputertechnik 1 · 90 Minuten · 90 Punkte

Musterlösung mit Bewertungshinweisen

Aufgabe 1: Bitmuster, Logik und Register (12 Punkte · 12 Minuten)

1. Wandeln Sie das achtbittige Muster `0xD6` in Binärdarstellung um. Geben Sie seinen Dezimalwert einmal ohne Vorzeichen und einmal im Zweierkomplement an. (3 P)
2. Erweitern Sie dasselbe Muster auf 32 Bit: einmal mit Vorzeichen, einmal mit Nullen. Schreiben Sie beide Ergebnisse hexadezimal. (2 P)
3. Addieren Sie achtbittig `0x6E` und `0x25`. Geben Sie das achtbittige Ergebnis, den ausgehenden Übertrag und den vorzeichenbehafteten Überlauf an. Begründen Sie den Unterschied zwischen den beiden letzten Angaben. (3 P)
4. Ein Multiplexer wählt bei $S = 0$ den Eingang A, bei $S = 1$ den Eingang B. Geben Sie seine Boolesche Gleichung für Y an und bestimmen Sie Y für $A = 1, B = 0$ bei beiden Auswahlwerten. (2 P)
5. Y liegt am Dateneingang eines positiv flankengetriggerten Ein-Bit-Registers. Anfangs ist $Q = 0$. Ergänzen Sie Q nach jedem Ereignis. Vor jeder steigenden Flanke liegen die Eingänge ausreichend lange stabil an. (2 P)

Ereignis	S	Schreibfreigabe E	Q danach
steigende Flanke	0	1	
S wechselt, keine neue Flanke	1	1	
steigende Flanke	1	0	
steigende Flanke	1	1	

Musterlösung

1. `1101 0110`, ohne Vorzeichen 214, als Zweierkomplement $214 - 256 = -42$. Je 1 P.
2. Mit Vorzeichen `0xFFFFFFD6`, mit Nullen `0x000000D6`. Je 1 P. Die Erweiterung entscheidet, ob die Bedeutung -42 oder 214 erhalten bleibt.
3. $110 + 37 = 147$, also `0x93` = `1001 0011`. Das Ergebnis bleibt unter 256, deshalb Übertrag 0. Als achtbittige vorzeichenbehaftete Summe ist 147 nicht darstellbar: Zwei positive Operanden erzeugen ein negatives Ergebnismuster (-109), also Überlauf 1. Je 1 P für Ergebnis, Übertrag und Überlauf mit Begründung.
4. $Y = (\overline{S} \wedge A) \vee (S \wedge B)$ (1 P). Bei $S = 0$ ist $Y = 1$, bei $S = 1$ ist $Y = 0$ (zusammen 1 P).
5. Die Folge ist $Q = 1, 1, 1, 0$ (je 0,5 P). Ohne aktive Flanke bleibt der Zustand erhalten. An der dritten Zeile verhindert $E = 0$ die Übernahme, obwohl die Flanke auftritt.

Aufgabe 2: RV32I codieren und decodieren (12 Punkte · 12 Minuten)

Es gelten RV32I ohne komprimierte Befehle, 32-Bit-Register, Byteadressierung und Little Endian. Echte Instruktionen sind vier Byte lang. Ein RISC-V-Referenzblatt darf verwendet werden.

1. Codieren Sie `addi x9, x8, -12`. Tragen Sie die Bitfelder ein und bilden Sie daraus das vollständige hexadezimale Maschinenwort. `addi` verwendet `funct3=000` und `opcode=0010011`. (4 P)

Feld	`imm[11:0]`	`rs1`	`funct3`	`rd`	`opcode`
Bitbreite	12	5	3	5	7
Inhalt binär					

2. Das Maschinenwort aus Teil 1 liegt ab `0x2800`. Geben Sie die vier Bytewerte mit ihren Adressen an. (2 P)
3. Decodieren Sie `0x40B50533` anhand der R-Felder. Bestimmen Sie Befehl, Zielregister und beide Quellen. Für `funct3=000` unterscheidet `funct7=0000000` Addition von `funct7=0100000` Subtraktion. (3 P)
4. Beurteilen Sie die drei Aussagen mit kurzer Begründung. (3 P)
   a. `addi x9, x8, -2049` ist unmittelbar als eine RV32I-Instruktion codierbar.
   b. `li x9, 5000` muss genau eine echte Instruktion ergeben.
   c. Nach `addi x0, x0, 12` liefert ein Lesen von `x0` zwölf.

Musterlösung

1. Das Zwölf-Bit-Immediate ist $4096 - 12 = 4084$, also `1111 1111 0100` = `0xFF4` (1 P). `rs1=01000`, `funct3=000`, `rd=01001`, `opcode=0010011` (zusammen 1 P). Verknüpfung:

   `111111110100 01000 000 01001 0010011`

   Das ergibt `0xFF440493` (2 P). Die beiden Quell-/Zielnummern sind fünf Bit breit; das I-Format hat kein zweites Quellregisterfeld.
2. `0x2800`: `0x93`, `0x2801`: `0x04`, `0x2802`: `0x44`, `0x2803`: `0xFF`. Je 0,5 P. Die Bytefolge ist von der üblichen Schreibweise des gesamten Maschinenworts zu unterscheiden.
3. `funct7=0100000`, `rs2=01011` (x11), `rs1=01010` (x10), `funct3=000`, `rd=01010` (x10), `opcode=0110011`. Ergebnis: `sub x10, x10, x11`, auch `sub a0, a0, a1`. Je 1 P für Operation, Ziel und Quellen.
4. Alle Aussagen sind falsch, je 1 P mit Begründung: (a) Der signed Zwölf-Bit-Bereich ist -2048 bis 2047. (b) `li` ist eine Pseudoinstruktion; 5000 passt nicht in ein `addi`-Immediate und benötigt eine passende Befehlsfolge. (c) `x0` liest immer null, der Schreibversuch wird verworfen.

Aufgabe 3: Speicher und Schleife verfolgen (14 Punkte · 14 Minuten)

Ein Array aus vier 32-Bit-Wörtern beginnt bei `s0=0x6200` und enthält die Werte 5, −3, 9 und 2. Der Speicher ist Little Endian. Ein fünftes gültiges Wort an `0x6210` ist für das Ergebnis reserviert. Alle Zugriffe sind ausgerichtet. Es gelten die RV32I-Annahmen aus Aufgabe 2.

```
    addi t0, s0, 0          # 0x0100: laufender Zeiger
    addi t1, x0, 4          # 0x0104: verbleibende Elemente
    addi t2, x0, 0          # 0x0108: Summe
loop:
    beq  t1, x0, done       # 0x010C
    lw   t3, 0(t0)          # 0x0110
    add  t2, t2, t3         # 0x0114
    addi t0, t0, 4          # 0x0118
    addi t1, t1, -1         # 0x011C
    jal  x0, loop           # 0x0120
done:
    sw   t2, 16(s0)         # 0x0124
```

1. Geben Sie die vier Bytes des Werts −3 mit Adressen an. Welche 32-Bit-Werte liefern `lb` und `lbu` von Adresse `0x6204`? (3 P)
2. Ergänzen Sie die Tabelle nach jedem Schleifenkörper. Geben Sie außerdem den schließlich gespeicherten Wert und seine Adresse an. (4 P)

Körperdurchlauf	geladener Wert	`t2` danach	`t0` danach	`t1` danach
1				
2				
3				
4				

3. Bestimmen Sie den Byteoffset des `beq` sowie den nächsten PC beim genommenen und beim nicht genommenen Branch. (2 P)
4. Wie viele echte Instruktionen werden von `0x0100` bis einschließlich `sw` dynamisch ausgeführt? Notieren Sie die Anteile für Initialisierung, Prüfungen, Schleifenkörper und Abschluss. (2 P)
5. Ersetzen Sie nur `addi t1, x0, 4` durch `addi t1, x0, 0`. Was wird gespeichert, wie viele Arrayelemente werden gelesen und wie viele Instruktionen werden ausgeführt? (3 P)

Musterlösung

1. −3 hat das Wortmuster `0xFFFFFFFD`: `0x6204` enthält `0xFD`, `0x6205` bis `0x6207` enthalten je `0xFF` (1 P). `lb` liefert `0xFFFFFFFD`, `lbu` liefert `0x000000FD` = 253 (je 1 P).

Körperdurchlauf	geladener Wert	`t2` danach	`t0` danach	`t1` danach
1	5	5	`0x6204`	3
2	−3	2	`0x6208`	2
3	9	11	`0x620C`	1
4	2	13	`0x6210`	0

2. Je Zeile 0,75 P. Gespeichert wird 13 an `0x6210` (1 P). Der Zeiger zeigt zuletzt hinter das Array, die folgende Prüfung verhindert einen fünften Load.
3. Offset $= \texttt{0x0124} - \texttt{0x010C} = \texttt{0x18} = 24$ Byte (1 P). Genommen: `0x0124`, nicht genommen: `0x0110` (zusammen 1 P). Der Offset bezieht sich auf die Adresse des Branch selbst, nicht auf PC + 4.

4. Drei Initialisierungen, fünf `beq`-Prüfungen, viermal fünf Körperinstruktionen einschließlich `jal` und ein abschließender Store: $N = 3 + 5 + 4 \cdot 5 + 1 = 29$ (2 P, anteilig). Auch der letzte genommene Abbruchbranch zählt, aber kein danach verworfener Befehl.
5. Es wird null an `0x6210` gespeichert (1 P). Kein Arrayelement wird gelesen (1 P). Ausgeführt werden $3+1+0+1 = 5$ Instruktionen (1 P). Der Store ist ein gültiger Zugriff auf den ausdrücklich reservierten Ergebnisplatz, kein Lesen eines Arrayelements.

Aufgabe 4: Aufruf und Stackframe reparieren (14 Punkte · 14 Minuten)

Die Funktion `f` soll $f(x) = 2x + 3$ berechnen. Dazu ruft sie `g` auf und braucht danach noch den ursprünglichen Wert von x. Es gelten RV32I und die Integer-Aufrufkonvention ILP32: Argument und Rückgabe in `a0`, `s0` muss erhalten bleiben, `t0` darf der Aufgerufene verändern. Der Stackzeiger bleibt auf 16 Byte ausgerichtet. Hier sind alle Ergebnisse darstellbar.

Beim Eintritt in `f` gelten `a0=7`, `s0=55`, `t0=17`, `sp=0x9000` und `ra=0x0144`. Der Bereich unter `sp` ist gültiger beschreibbarer Speicher. Jede aufgeführte Zeile belegt vier Byte, auch `ret` als Alias für `jalr x0, 0(ra)`.

```
f:
    addi sp, sp, ____          # 0x0200  (a)
    sw   ra, 12(sp)            # 0x0204
    sw   s0, ____(sp)          # 0x0208  (b)
    addi s0, a0, 0             # 0x020C
    jal  ra, g                 # 0x0210
    add  a0, a0, ____          # 0x0214  (c)
    lw   s0, ____(sp)          # 0x0218  (d)
    lw   ra, 12(sp)            # 0x021C
    addi sp, sp, ____          # 0x0220  (e)
    ret                       # 0x0224
g:
    addi a0, a0, 3             # 0x0300
    addi t0, x0, 99            # 0x0304
    ret                       # 0x0308
```

1. Füllen Sie die fünf Lücken mit einem gemeinsamen 16-Byte-Frame für die beiden Sicherungen. (5 P)
2. Geben Sie `a0`, `s0`, `t0`, `sp` und `ra` nach der Rückkehr von `f` an. Wohin springt das letzte `ret`? (3 P)
3. Welche Werte liegen nach den beiden Stores an welchen Wortadressen im Frame? (2 P)
4. Welchen Linkwert schreibt `jal ra, g`, und welches Ziel hat das `ret` in `g`? (2 P)
5. Warum muss `f` `ra` sichern? Warum kann sie den ursprünglichen Wert von x hier nicht ungesichert in `t0` aufbewahren? (2 P)

Musterlösung

1. (a) −16, (b) 8, (c) `s0`, (d) 8, (e) 16. Je 1 P. Die Sicherungen belegen verschiedene Slots; die übrigen acht Byte sind Füllung zur Ausrichtung.
2. `a0=17`, `s0=55`, `t0=99`, `sp=0x9000`, `ra=0x0144`; Rückkehrziel `0x0144`. Je Angabe 0,5 P. `g` liefert $7+3 = 10$, `f` addiert den bewahrten Eingang 7. Der Inhalt von `t0` ist hier durch den gezeigten Helfer bekannt, muss aber nach der Konvention grundsätzlich als verändert betrachtet werden.
3. Neuer Stackzeiger `0x8FF0`. An `0x8FFC` liegt die äußere Rückkehradresse `0x0144`, an `0x8FF8` der alte `s0`-Wert 55. Je Wert mit Adresse 1 P. `s0` wird erst nach seiner Sicherung mit 7 überschrieben.
4. Linkwert `0x0210` + 4 = `0x0214` (1 P). Das `ret` in `g` springt nach `0x0214` (1 P), nicht direkt zum Aufrufer von `f`.
5. Der innere `jal` überschreibt `ra`; ohne Sicherung wäre die äußere Rückkehradresse verloren (1 P). `t0` ist caller-saved und wird von `g` tatsächlich auf 99 gesetzt; ohne Sicherung entstünde $10 + 99 = 109$ statt 17 (1 P). Das verwendete `s0` bleibt über `g` erhalten und wird von `f` vor ihrer eigenen Rückkehr restauriert.

Aufgabe 5: Datenpfad und Steuersignale (14 Punkte · 14 Minuten)

Es gilt der erweiterte Single-Cycle-Lehrdatenpfad aus Vorlesung 5/6. Die Recheneinheit ALU addiert für `lw`, `sw` und `addi`; bei `beq` subtrahiert sie für den Vergleich. `ALUSrc=1` wählt das Immediate statt Registeroperand 2. `MemtoReg=1` wählt gelesene Daten statt ALU-Ergebnis zum Rückschreiben. `RegWrite`, `MemRead` und `MemWrite` geben die benannten Schreib-/Lesezugriffe frei. `Branch` kennzeichnet `beq`. Die Immediate-Erzeugung liefert bereits den fertigen vorzeichenerweiterten Byteoffset; ein weiteres Verdoppeln findet nicht statt.

Anfang: `s1=0x7400`, `t2=3`. Das gültige Wort an `0x740C` enthält 26; das Wort an `0x7410` ist beschreibbar.

```
lw   t2, 12(s1)          # 0x0400
addi t3, t2, -6          # 0x0404
sw   t3, 16(s1)          # 0x0408
beq  t3, t2, ziel        # 0x040C
                        # Fall-through: 0x0410
                        # ziel:         0x0420
```

1. Ergänzen Sie die Steuertabelle. `X` ist nur zulässig, wenn beide Auswahlwerte nachweislich dieselbe sichtbare Wirkung haben. (6 P)

Befehl	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch
`lw`						
`addi`						
`sw`						
`beq`						

2. Verfolgen Sie den Ausschnitt: Bestimmen Sie pro Befehl beide ALU-Eingangswerte, das ALU-Ergebnis, die Register-/Speicheränderung und den nächsten PC. (4 P)
3. Geben Sie die Gleichung für `PCSrc` an. Welcher nächste PC ergäbe sich, wenn nur der Branch durch `beq t3, t3, ziel` ersetzt würde? (2 P)
4. Bei `lw` ist irrtümlich `MemtoReg=0` verdrahtet, alle anderen Signale sind richtig. Welcher Wert landet nun in `t2` und welcher im Ergebniswort an `0x7410`? (2 P)

Musterlösung

Befehl	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch
`lw`	1	1	1	1	0	0
`addi`	1	0	1	0	0	0
`sw`	1	X	0	0	1	0
`beq`	0	X	0	0	0	1

1. Je Zeile 1,5 P, anteilig. Bei `sw` und `beq` ist statt X auch 0 oder 1 zulässig: `RegWrite=0` verhindert, dass die Auswahl den Registerzustand ändert. Schreibfreigaben sind dagegen keine beliebigen Don't-care-Werte.

Befehl	ALU-Eingang A	ALU-Eingang B	ALU-Ergebnis	Änderung	nächster PC
`lw`	`0x7400`	12	`0x740C`	`t2=26`	`0x0404`
`addi`	26	−6	20	`t3=20`	`0x0408`
`sw`	`0x7400`	16	`0x7410`	Wort dort = 20	`0x040C`
`beq`	20	26	−6 = `0xFFFFFFFA`	keine	`0x0410`

2. Je Zeile 1 P, anteilig. Store-Daten kommen aus `t3` und nicht vom ALU-Eingang B; dieser enthält den Adressoffset. Beim Load ist das ALU-Ergebnis eine Adresse und der Rückschreibwert deren Inhalt.
3. PCSrc $=$ Branch $\wedge$ Zero (1 P). Mit gleichen Operanden ist Zero 1 und der nächste PC `0x0420` (1 P): `0x040C` $+ 20 =$ `0x0420`.
4. Statt der geladenen 26 wird die ALU-Adresse `0x740C` nach `t2` geschrieben (1 P). Danach erzeugt `addi` den Wert `0x7406` = 29 702; der Store schreibt diesen nach `0x7410` (1 P). Die Speicheradresse wird weiterhin richtig gebildet.

Aufgabe 6: Aus einer Tabelle wird Steuerlogik (10 Punkte · 10 Minuten)

Ein Vorerkenner liefert einen frei gewählten Gruppencode $g_1 g_0$. Dieser Code gehört nur zum Aufgabenmodell und ist kein Feld der RISC-V-Instruktionscodierung:

$g_1 g_0$	Befehl
00	`addi`
01	`lw`
10	`sw`
11	`beq`

1. Bestimmen Sie die vier Freigaben. Ihre Bedeutung entspricht Aufgabe 5. (4 P)

$g_1 g_0$	RegWrite	MemRead	MemWrite	Branch
00				
01				
10				
11				

2. Leiten Sie für diese vier Ausgänge möglichst einfache Boolesche Gleichungen in g_1 und g_0 her. (4 P)
3. Ein anderer Teil der ALU-Steuerung behandelt die Registerbefehle `add`, `sub`, `and` und `or`. Ein Team möchte deren Operation allein durch Bit 30 des Maschinenworts festlegen: 0 für Addition, 1 für Subtraktion. Warum genügt das nicht für alle vier Befehle? Welche zusätzlichen Informationen müssen ausgewertet werden? (2 P)

Musterlösung

$g_1 g_0$	RegWrite	MemRead	MemWrite	Branch
00	1	0	0	0
01	1	1	0	0
10	0	0	1	0
11	0	0	0	1

1. Je Zeile 1 P, anteilig. Nur `addi` und `lw` schreiben ein Register. Jeder Speicherzugriff erhält ausschließlich seine passende Freigabe.
2. Je 1 P:

$$\text{RegWrite} = \overline{g_1}, \qquad \text{MemRead} = \overline{g_1} \wedge g_0$$

$$\text{MemWrite} = g_1 \wedge \overline{g_0}, \qquad \text{Branch} = g_1 \wedge g_0$$

   Gleichwertige, noch nicht vereinfachte Gleichungen sind ebenfalls korrekt. Das Setzen eines Ausgangs muss genau die passenden Tabellenzeilen treffen.
3. `add`, `and` und `or` haben alle Bit 30 gleich null, führen aber verschiedene Operationen aus (1 P). Innerhalb der ausgewiesenen Registeroperationen unterscheiden die Felder `funct3` und `funct7` die Befehle: bei `funct3=000` Addition/Subtraktion, bei `111` UND, bei `110` ODER. Die Hauptsteuerung muss zuvor die passende Befehlskategorie erkennen (1 P). Die interne Codierung von ALU-Steuerleitungen ist keine Vorgabe der Befehlssatzarchitektur.

Aufgabe 7: Pipeline, Laufzeit und Vorhersage (14 Punkte · 14 Minuten)

Es gilt die fünfstufige Kurspipeline: IF (Holen), ID (Decodieren/Registerlesen), EX (Rechnen), MEM (Speicher), WB (Rückschreiben). Sie ist anfangs leer; gezählt wird von erstem IF bis letztem WB. Getrennte Instruktions-/Datenzugriffe verursachen keine Strukturkonflikte. EX/MEM und MEM/WB leiten verfügbare Ergebnisse nach EX weiter; ein Load liefert seine Daten erst nach MEM. Bei einem Stall halten PC und IF/ID, während der Produzent weiterläuft und eine wirkungslose Bubble nach EX gelangt. Ein in WB geschriebener Wert ist erst im Folgetakt in ID lesbar; ein WB→ID-Bypass fehlt.

Für Teil 1–3 gelten `s0=0x8100`, Wort dort = 12, `s1=5`, `a1=2`. Die übrigen verwendeten Register dürfen vorher beliebige Werte enthalten, die Pipeline behandelt Abhängigkeiten korrekt.

```
lw   t0, 0(s0)            # I1
sub  t1, t0, s1           # I2
addi a1, a1, 1            # I3
xor  a0, t1, s1           # I4
```

1. Zeichnen Sie das Stufenraster einschließlich Bubble; kennzeichnen Sie gehaltene Stufen. Wie viele Takte braucht die Folge? (4 P)
2. Geben Sie `t0`, `t1`, `a1` und `a0` nach Abschluss an. Aus welchen Pipeline-Registern stammen `t0` für I2 und `t1` für I4 jeweils in EX? (2 P)
3. Ordnen Sie die vier Instruktionen ohne Änderung ihrer Wirkung so um, dass die Folge einen Takt spart. (2 P)
4. Eine unabhängige neue Folge besteht aus `add t0,t2,t3`, `addi a1,a1,1`, `xor a2,a2,a2`, `sub a0,t0,s1`. Alle benötigten Anfangswerte stehen bereit. Warum reicht EX-Forwarding hier allein nicht, und wie viele Takte braucht diese Folge im Kursmodell? (2 P)
5. Eine andere Routine beendet $N = 80$ Instruktionen. Es treten fünf Daten-Stalltakte und vier genommene Branches auf; jeder Branch verwirft wegen Auflösung in EX zwei jüngere Befehle. Die Effekte überlappen nicht. Angenommene Lehrwerte eines anderen Entwurfs: Pipeline $T = 240$ ps, Single Cycle $T = 900$ ps und CPI 1. Berechnen Sie Pipeline-Taktzahl, Pipeline-Laufzeit und Speedup gegenüber Single Cycle. (2 P)
6. Ein Ein-Bit-Prädiktor für denselben Branch startet mit N (nicht genommen), sagt seinen Zustand voraus und übernimmt anschließend das tatsächliche Ergebnis. Die Ergebnisse sind T, T, N, T (T = genommen). Notieren Sie die Vorhersagen und die Positionen der Fehlvorhersagen. (2 P)

Musterlösung

Eintrag	1	2	3	4	5	6	7	8	9
I1	IF	ID	EX	MEM	WB				
I2		IF	ID	ID*	EX	MEM	WB		
I3			IF	IF*	ID	EX	MEM	WB	
I4					IF	ID	EX	MEM	WB
Bubble				EX	MEM	WB			

1. Ein Load-Use-Haltezyklus: I2 kann die Daten nicht schon in EX-Takt 4 nutzen. Der Produzent I1 läuft weiter, I2 bleibt in ID, I3 bleibt in IF und die Bubble durchläuft EX/MEM/WB (3 P für das Raster, anteilig). Gesamt $C = 4 + 4 + 1 = 9$ Takte (1 P).
2. `t0=12`, `t1=7`, `a1=3`, `a0=2`, denn $0111_2 \oplus 0101_2 = 0010_2$ (1 P). I2 erhält `t0` in EX-Takt 5 aus MEM/WB von I1. I4 erhält `t1` in EX-Takt 7 aus MEM/WB von I2 (zusammen 1 P). In ID wurden gegebenenfalls alte Registerwerte gelesen, die Auswahl am Recheneingang ersetzt sie.
3. Reihenfolge I1, I3, I2, I4 (1 P). Der unabhängige Zähler belegt den zuvor verlorenen Abstand. I2 erhält den Loadwert nach MEM, I4 den Wert von I2 aus EX/MEM; $C = 4 + 4 = 8$ (1 P). Keine Datenabhängigkeit verbietet die Verschiebung.
4. Der erste `add` ist in Takt 5 in WB, der letzte `sub` gleichzeitig in ID und liest dort noch den alten `t0`-Wert (1 P). In Takt 6 befindet sich der erste Produzent bereits außerhalb der Forwarding-Register. Daher hält `sub` einen Takt in ID und liest in Takt 6 erneut; EX folgt in Takt 7, WB in Takt 9. Gesamt neun Takte (1 P). Ein zusätzlicher WB→ID-Bypass würde ein anderes Modell definieren.
5. $C = N+4+H+2B = 80+4+5+2\cdot 4 = 97$ und $t_{\text{Pipe}} = 97 \cdot 240 = 23\,280$ ps $= 23{,}28$ ns (1 P). $t_{\text{Single}} = 80 \cdot 900 = 72\,000$ ps $= 72$ ns, Speedup $S = 72/23{,}28 \approx 3{,}09$ (1 P). Die verworfenen Befehle zählen nicht zusätzlich zu den 80 abgeschlossenen Instruktionen.
6. Vorhersagen N, T, T, N (1 P). Fehler bei 1, 3 und 4 (1 P). Der Endzustand ist T. Ein einzelner Zustandswechsel korrigiert erst die nächste Vorhersage, nicht die bereits getroffene Entscheidung.